

Article

Optimal Allocations Under Strongly Pigou–Dalton Criteria: Hidden Layer Structure and Efficient Combinatorial Approach

Taikun Zhu , Kai Jin ^{*} , Ruixi Luo  and Song Cao 

School of Intelligent Systems Engineering, Shenzhen Campus, Sun Yat-sen University, Shenzhen 518107, China; zhutk3@mail2.sysu.edu.cn (T.Z.); luorx@mail2.sysu.edu.cn (R.L.); caos6@mail2.sysu.edu.cn (S.C.)

* Correspondence: jink8@mail.sysu.edu.cn

Abstract

We investigate optimal social welfare allocations of m items to n agents with binary additive or submodular valuations. For binary additive valuations, we prove that the set of optimal allocations coincides with the set of so-called *stable allocations*, as long as the employed criterion for evaluating social welfare is strongly Pigou–Dalton (SPD) and symmetric. Many common criteria are SPD and symmetric, such as Nash social welfare, LexiMax, LexiMin, the Gini Index, Entropy, and Envy Sum. We also design efficient algorithms for finding a stable allocation, including an $O(m^2n)$ time algorithm for the case of indivisible items, and an $O(m^2n^5)$ time one for the case of divisible items. The first is faster than the existing algorithms or has a simpler analysis. The latter is the first combinatorial algorithm for that problem. It utilizes a hidden layer partition of items and agents admitted by all stable allocations, and cleverly reduces the case of divisible items to the case of indivisible items. In addition, we show that the profiles of different optimal allocations have a small Chebyshev distance, which is zero for the case of divisible items under binary additive valuations, and is at most one for the case of indivisible items under binary submodular valuations.

Keywords: optimal social welfare; strongly Pigou–Dalton; binary additive or submodular valuation; combinatorial algorithm; layer partition

MSC: 91B32; 91B08; 90C27

1. Introduction

Maximizing social welfare or minimizing inequality in allocating resources to agents is an important topic in social economics and has been studied extensively in recent years [1–4]. Each agent has her own subjective *valuation function* over a subset of resources (items). Yet how to evaluate the welfare or inequality has no unified answer. Some may suggest LexiMin as the criterion [5]—where we maximize the smallest valuations of all agents, then maximize the second smallest valuations of them, and so on; while others may suggest Maximum Nash Welfare (MNW) [4], where we maximize the product of valuations of all agents. For most classes of valuation functions, such as the additive valuation functions (where an agent’s valuation is the sum of her valuations for individual items in her bundle), the optimal allocations vary with the selected criteria.

For the special case of *binary additive valuations* on items, however, Aziz and Rey [6] showed that the LexiMin allocations are equivalent to the MNW allocations. This result raises a natural question of whether there are more connections among other criteria.



Academic Editor: Jun Zhang

Received: 2 January 2026

Revised: 6 February 2026

Accepted: 10 February 2026

Published: 12 February 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Benabbou et al. [7] gave a positive answer—the optimal allocations under any *strongly Pigou–Dalton (SPD)* principle (a.k.a. transfer principle) [8] over all allocations with maximal utilitarian social welfare (USW, defined as the sum of valuations, a promise of efficiency) are consistent with LexiMin allocations, even under *binary submodular valuations*, which subsume binary additive valuations. Roughly, the Pigou–Dalton principle requires that if some income is transferred from a rich person to a poor person, while not bringing the rich to a poorer situation than the poor, then the measured inequality should not increase (or decrease in the strong version); see its formal definition in Section 2. The SPD principle embodies a strictly egalitarian stance: any feasible redistribution toward balance must be reflected in a measurable welfare improvement. It is particularly compelling in the context where society prioritizes minimal disparity over efficiency margins. Most common criteria admit the SPD principle (see Example 1).

Inspired by the aforementioned consistency of optimums among all the SPD criteria [7], we conduct a further study of the optimums under the SPD criteria mainly for the following three scenarios:

IND. Indivisible items and agents with binary additive valuations.

DIV. Divisible items and agents with binary additive valuations.

IND-SUB. Indivisible items and agents with binary submodular valuations.

Our main results are summarized below. We are only concerned with the allocations with maximal USW to ensure the efficiency, like in [7]. Unless otherwise stated, the criteria for evaluating inequality are SPD and symmetric. Moreover, a profile of an allocation χ refers to the valuations vector of the n agents under χ :

- (a) For IND, the profiles of optimal allocations have a Chebyshev distance of at most *one*. Moreover, there is a *layer structure* hidden behind the optimal allocations—the agents and items are partitioned into several layers so that items can only be allocated to the agents within the same layer in all optimal allocations. The layer structure admits the property that within each layer, the number of items allocated to any two agents differs by at most *one*—a fixed and small constant that is independent of a specific instance.
- (b) In DIV, the profiles of optimal allocations are all the same (in other words, the Chebyshev distance is *zero*). Halpern et al. [9] have shown a similar result in which only LexiMin and MNW are considered.
A layer structure still exists (and with more layers compared to the scenario of IND). More importantly, by utilizing this layer structure and using a reduction to IND, we design the first combinatorial algorithm for finding the optimal allocation for DIV, which runs in $O(m^2n^5)$ time.
- (c) In IND-SUB, the profiles of optimal allocations have a Chebyshev distance of at most *one*. This extends the corresponding result in IND. However, the layer structure mentioned above does not hold under this scenario.

See Table 1 for a summary of the results. Our research objects are mainly the optimal allocations under any SPD criteria. In all cases, we derive an “almost consistency” among different optimums (SPD criteria). It states that the valuation of any agent differs by at most *one* (or *zero* for the DIV case), under any two different optimal allocations. We further discover layer structures for the IND and DIV scenarios and design the first combinatorial algorithm for DIV based on the layer structure.

Table 1. A summary of the results.

Scenario	SPD Criteria Consistency	Running Time	Cheby. Dist.	Layer Structure
IND	Yes	$O(m^2n)$	≤ 1	Yes
DIV	Yes	$O(m^2n^5)$	0	Yes
IND-SUB	Yes [7]	poly [10]	≤ 1	No

Related Work

Halpern et al. [9] show that under binary additive valuations, given any *fractional MNW allocation* (i.e., the MNW allocations in the scenario of DIV), one can compute, in polynomial time, a *randomized allocation* with only *deterministic MNW allocations* (i.e., the MNW allocations in the scenario of IND) in its support and the randomized allocation implements the given fractional MNW allocation. This is a compelling connection between the deterministic and fractional MNW allocations: given a fractional MNW allocation, one can find a convex combination of deterministic MNW allocations to yield it. We note that the connection found by them is not a computational method for fractional MNW allocation (since they need a fractional MNW allocation as the input), and our method finds an optimal allocation of the scenario of DIV with an algorithm designed for computing optimal allocations of the scenario of IND.

For computational tractability, the SPD optimal allocations can be computed in polynomial time (by computing a LexiMin or MNW allocation) in the scenario of IND [2,4,5], DIV [11] and IND-SUB [6]. We propose a method to find an optimal allocation of the scenario of DIV with an algorithm designed for computing optimal allocations of the scenario of IND. This is reminiscent of the relation between integer programming and linear programming. The well-known branch-and-bound method uses linear programming as a subprogram to solve the integer programming problem. In this paper, based on nontrivial observations, we split each item into a fixed number of pieces and prove that the optimal allocation over the pieces (viewed as indivisible items) is exactly an optimal allocation of the scenario of DIV.

The setting of binary valuation is considered in the resource allocation problem, optimal job scheduling, and load balancing problem. Lin and Li [12] study the special case in which each job can be processed on a subset of allowed machines, and its run-time in each of these machines is one, and find the minimum makespan in polynomial time. Kleinberg et al. [5] study a case called uniform load balancing, which is to assign jobs to machines so that the set of allocated bandwidth is LexiMin. The objective is to find the allocation that the numbers of jobs assigned to each machine is LexiMax optimal: lexicographically minimizing the number of jobs assigned to each machine when sorted from largest to smallest.

Another classic class of resource allocation problems is that with only one kind of resource. In this setting, we only care about the number of items allocated to each agent, rather than the specific subset of items. Ibaraki and Katoh [13] make a review on this class of resource allocation problem, viewing it as an optimization problem. They allocate a fixed amount of resources (continuous or discrete) to n agents for optimizing the objective function (e.g., separable, convex, minimax, or general). In particular, an important case is the discrete resource allocation problem with a separable convex objective function, where Michaeli and Pollatschek [14] discuss some properties between the optimal solution of the discrete version and that of the continuous version. These properties can be used to speed up the search for an integer solution.

The resource allocation problems under ternary valuations, which are a natural extension of binary valuations, are much harder to handle. Under additive valuations,

Golovin [15] proves that it is NP-hard to compute a $(2 - \epsilon)$ approximate maximin allocation even with agents' valuations of $\{0, 1, 2\}$ on single items. Another extension of binary valuations is the case where item j has value p_j or zero for each agent. In this case of valuations, for computing maximin allocation, Bezakova and Dani [16] prove that there is no approximation algorithm with a performance guarantee better than 2 unless $P = NP$, and Bansal and Sviridenko [17] present an $O(\frac{\log \log m}{\log \log \log m})$ approximation algorithm.

2. Preliminaries

For an integer $k > 0$, let $[k]$ denote $\{1, \dots, k\}$. Throughout this paper, $[m]$ refers to the set of m items and $[n]$ refers to the set of n agents.

Each agent $i \in [n]$ has a valuation function $v_i : 2^{[m]} \rightarrow \mathbb{R}_+$ over subsets of $[m]$ (called bundles), where $v_i(\emptyset) = 0$. Given a valuation function v_i , we define the *marginal gain* of an item o over a bundle $S \in [m]$ as $\Delta_i(S; o) = v_i(S \cup o) - v_i(S)$. We focus on the binary valuations where the marginal gain $\Delta_i(S; o) \in \{0, 1\}$.

Two kinds of binary valuations are discussed frequently in the literature, which we call **0/1-add** and **0/1-sub**. For the 0/1-add valuations, the value of a set of items for an agent is the sum of the valuation of the individual items; the marginal gain $\Delta_i(S; o)$ is based on whether agent i likes item o (and is independent of S). For the 0/1-sub valuations, the marginal gain $\Delta_i(S; o)$ does not increase when S grows; formally, $\Delta_i(T; o) \leq \Delta_i(S; o)$ for $S \subset T \subset [m]$ and $o \in [m] \setminus T$.

Note that 0/1-sub valuations subsume 0/1-add ones.

An allocation χ refers to a collection of disjoint bundles $\chi_1 \dots \chi_n$, such that $\chi_1 \cup \dots \cup \chi_n \subseteq [m]$. An allocation χ is *clean* if all the bundles are clean— χ_i is clean if it has no items with zero marginal gain (i.e., for all $o \in \chi_i$, $\Delta_i(\chi_i \setminus \{o\}; o) = 1$). For 0/1-sub valuations, χ_i is clean if and only if $v_i(\chi_i) = |\chi_i|$ (Proposition 3.3 of [7]).

Given a clean allocation χ , assuming agent i gets $h_i = |\chi_i|$ items under χ , we call vector $(h_1, \dots, h_n)$ the *profile* of χ , denoted by $\mathbf{p}(\chi)$. Henceforth, h_i always refers to $|\chi_i|$ unless otherwise stated.

Definition 1 (Criterion). A criterion of income inequality (criterion for short), a.k.a. an income inequality metric [1,3], is a function from the profiles to $\mathbf{R}$: Each profile is evaluated by a real number (called score); the lower the score, the better the profile under this criterion. Following the convention, a criterion must be symmetric; i.e., it should evaluate all permutations of $\mathbf{p}$ equally.

Example 1 (Some commonly used criteria). Let $h_1^\uparrow, \dots, h_n^\uparrow$ be the permutation of $h_1, \dots, h_n$, sorted in increasing order. Take $\Phi(x)$ to be any strictly convex function of x . For example, $\Phi(x) = x^2$. For every profile $\mathbf{p} = (h_1, \dots, h_n)$.

$$\begin{aligned} \text{NSW}^\top(\mathbf{p}) &:= - \prod_{h_i > 0} h_i; & \text{Potential}_\Phi(\mathbf{p}) &:= \sum_{i=1}^n \Phi(h_i); \\ \text{GiniIndex}(\mathbf{p}) &:= \sum_i i \cdot h_i^\uparrow; & \text{EnvySum}(\mathbf{p}) &:= \sum_{h_i < h_j} (h_j - h_i); \\ \text{Congestion}(\mathbf{p}) &:= \sum_i \binom{h_i}{2}; & \text{Entropy}^\top(\mathbf{p}) &:= \sum_i \frac{h_i}{m} \log\left(\frac{h_i}{m}\right); \\ \text{LexiMax}(\mathbf{p}) &:= \sum_i m^{h_i}; & \text{LexiMin}(\mathbf{p}) &:= \sum_i m^{m-h_i}. \end{aligned}$$

Remark 1. For $\text{NSW}^\top$, we first need to maximize the number of agents with nonzero valuation and then maximize the product of the nonzero valuations. By the last two definitions, minimizing LexiMax is equivalent to minimizing the largest valuation of all agents, then minimizing the second

largest valuation of them, and so on; minimizing LexiMin is equivalent to maximizing the smallest valuation of all agents, then maximizing the second smallest valuation of them, and so on.

We abbreviate $f(\mathbf{p}(\chi))$ as $f(\chi)$ for any criterion f .

Definition 2 (Strongly Pigou–Dalton). Profile $\mathbf{q} = (q_1, \dots, q_n)$ is regarded more balanced than $\mathbf{p} = (p_1, \dots, p_n)$, if there are $j, k \in [n]$, such that both q_j, q_k lie in range (p_j, p_k) , $p_j + p_k = q_j + q_k$, and $q_i = p_i$ for $i \in [n] \setminus \{j, k\}$ (namely, the incomes of two agents are more balanced in $\mathbf{q}$, whereas all other incomes remain unchanged). A criterion f is strongly Pigou–Dalton (SPD) [8] if $f(\mathbf{q}) < f(\mathbf{p})$ whenever $\mathbf{q}$ is more balanced than $\mathbf{p}$. (The SPD principle is also known as the transfer principle.)

All criteria shown in Example 1 are SPD (proved in Appendix A).

We only consider the allocation with maximal utilitarian social welfare (max-USW, maximizing the sum of the valuations of all agents); otherwise, one may minimize LexiMax(χ) (GiniIndex(χ), etc.) by not allocating any items, which is uninteresting.

Henceforth, unless otherwise stated, allocations are assumed to be **max-USW** and **clean** (to this end, we can drop items with zero marginal gain without changing the profile).

3. Indivisible Items and Agents with 0/1-Add Valuations

Definition 3 (Stable allocations in the IND scenario). Take an allocation χ of indivisible items. For each item o allocated to agent i that can be reallocated to another agent i' (i.e., $v_{i'}(\{o\}) = 1$), build an edge (i, i') . Moreover, if there is a simple path $(i_1, \dots, i_k)$ ($k \geq 2$) along such edges, we state that χ admits a transfer from $i_1 = u$ to $i_k = v$, denoted by $u \rightarrow v$, which consists of $k - 1$ reallocations along the path, after which agent u loses and v gains one.

A narrowing transfer refers to a transfer $u \rightarrow v$ with $h_u \geq h_v + 2$. A widening transfer refers to a transfer $u \rightarrow v$ with $h_u \leq h_v$. Other transfers (i.e., $u \rightarrow v$ with $h_u = h_v + 1$) are called swapping transfers.

Allocation χ is called nonstable if it admits a narrowing transfer, and is called stable otherwise. Denote by S the set of stable allocations.

Barman et al. [4] proved that if an allocation is not optimal under $\text{NSW}^\top$, then it admits a narrowing transfer (hence it is nonstable). Their result can be easily generalized to any SPD criterion, including LexiMin (as stated in Lemma 1).

Lemma 1. Stable allocations are optimal under LexiMin.

(As a corollary, their profiles are equivalent under permutation.)

Proof. Assume χ is non-optimal under LexiMin. We shall prove that χ is nonstable, i.e., it admits a narrowing transfer.

First, take an allocation χ^* that is optimal under LexiMin.

We build a graph G with n vertices. If an item is allocated to j in χ and allocated to k in χ^* , where $k \neq j$, build an arc from j to k . Note that G may have duplicate arcs. Clearly, an arc represents a reallocation (of one item) on χ , and χ becomes χ^* after all the arcs (i.e., reallocations) are applied.

We decompose G into several cycles $C_1, \dots, C_a$ and paths $P_1, \dots, P_b$. Denote by s_i, t_i the starting and ending vertices of P_i , respectively. We assume that $t_j \neq s_k$ for any $j \neq k$; otherwise we connect the two paths P_j, P_k into one path.

For $0 \leq i \leq b$, let $\chi^{(i)}$ be the allocation copied from χ but applied all the arcs (reallocations) in $P_1, \dots, P_i$. $\chi^{(0)} = \chi$.

Note that $\chi^{(b)}$ becomes χ^* after applying the arcs in $C_1, \dots, C_a$. We obtain that $\text{LexiMin}(\chi^{(b)}) = \text{LexiMin}(\chi^*)$. Further since that $\text{LexiMin}(\chi^*) < \text{LexiMin}(\chi)$, there exists i ($1 \leq i \leq b$), such that $\text{LexiMin}(\chi^{(i)}) < \text{LexiMin}(\chi^{(i-1)})$.

It follows that in $\chi^{(i-1)}$, we have $h_s \geq h_t + 2$ (where s, t denote s_i, t_i respectively, for short). It further follows that in $\chi^{(0)} = \chi$, we also have $h_s \geq h_t + 2$, as h_s never increases and h_t never decreases in the sequence $\chi^{(0)}, \dots, \chi^{(i-1)}$. Consequently, χ admits a narrowing transfer (from s to t). $\square$

Theorem 1. 1. For any SPD criterion, the optimums are exactly $\mathcal{S}$.
2. We can find a stable allocation in $O(m^2n)$ time.

Proof. 1. Fix any SPD criteria f ; we need to show that

- (1) A nonstable allocation χ is non-optimal under f .
- (2) A stable allocation χ is optimal under f .

Together, the optimal allocations under f are the stable ones.

Proof of (1): If χ is nonstable, it admits a narrowing transfer; denoted by χ' is the allocation after this transfer. Clearly, $\mathbf{p}(\chi')$ is more balanced than $\mathbf{p}(\chi)$, and therefore $f(\chi') < f(\chi)$ due to the assumption that f is strongly Pigou–Dalton.

Proof of (2): Assume χ, χ' are stable. By Lemma 1, $\mathbf{p}(\chi)$ is equivalent to $\mathbf{p}(\chi')$ up to permutation; therefore, $f(\chi) = f(\chi')$ as f is symmetric (a reminder that we always assume so). So, stable allocations have the same score under f . Further since that nonstable allocations are non-optimal under f ((1) of Claim 1), all stable allocations admit the same lowest score under f (a.k.a. optimal).

2. Finding a stable allocation reduces to finding the allocation with minimum Congestion (by Claim 1 of this theorem), which can be found using network flows. Specifically, it reduces to computing the minimum-cost flow in the following network (see Figure 1).

There are $m + n + 2$ nodes, including a source node s , a sink node t , and m nodes $u_1, \dots, u_m$ representing the items and n nodes $v_1, \dots, v_n$ representing the agents. And there are $\Theta(mn)$ edges in the network: (i) an edge from s to each u_i , with capacity one and cost zero; (ii) an edge from u_i to v_j if agent j likes item i , with capacity one and cost zero; and (iii) m edges from each v_j to t , in which the k th one has capacity one and cost $k - 1$.

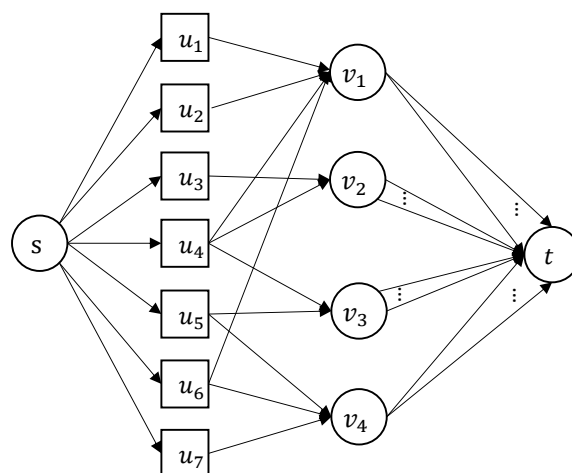


Figure 1. Reduction to minimum-cost flow problem.

Our target—a flow of size m with the minimum cost—can be computed by the Successive Shortest Path algorithm [18], which increases the size of the current flow by one via augmenting along the shortest path in the residual graph, repeating m times. For our particular network, finding such a path reduces to finding a non-used edge (v_j, t) with the

lowest cost, such that s can reach v_j in the residual network, which can be done in $O(mn)$ time by BFS. In total it runs in $O(m^2n)$ time. $\square$

Our Algorithm 1 is similar to that of Kleinberg et al. [5] for finding a LexiMax optimal allocation, but our analysis is simpler.

Algorithm 1 STABLEALLOCATIONIND($[m], [n], B$)

Input: m items $[m]$, n agents $[n]$, binary valuation matrix B of size $n \times m$, where $B(i, j)$ denotes the valuation of item i for agent j .

Output: A stable allocation $\chi = (\chi_1, \dots, \chi_n)$

- 1: Construct flow network $G = (V, E)$ with nodes $\{s, t\} \cup \{u_1, \dots, u_m\} \cup \{v_1, \dots, v_n\}$.
 - 2: Add edge (s, u_i) with capacity one and cost zero for each item $i \in [m]$.
 - 3: Add edge (u_i, v_j) with capacity one and cost zero if $B(i, j) = 1$ (agent j likes item i).
 - 4: Add m edges from each v_j to t , where the k th edge has capacity one and cost $k - 1$.
 - 5: Compute min-cost flow f of value m using SUCCESSIVESHORTESTPATH [18].
{Running time: $O(m^2n)$ }
 - 6: Add item i to χ_j for each flow $f(u_i, v_j) = 1$.
 - 7: **return** χ
-

Remark 2. It might be asked whether a better allocation under some criterion is also a better allocation under another criterion? This answer is no, although the best remains the best crossing different criteria (Theorem 1). For example, for $m = 14$ and $n = 3$, we have

$$\text{Congestion}((0, 5, 9)) = 46 < 47 = \text{Congestion}((2, 2, 10)), \text{ and} \\ \text{EnvySum}((0, 5, 9)) = 18 > 16 = \text{EnvySum}((2, 2, 10)).$$

3.1. Layer Partition of Agents and Items

Recall that the profiles of stable allocations are equivalent up to permutation (see the corollary part in Lemma 1). Yet the profiles are not unique—e.g., the number of items h_i allocated to agent i may differ in different stable allocations. It raises a natural question: to what extent can $\mathbf{p}(\chi)$ differ for different χ in $\mathcal{S}$?

Our next theorem shows that the difference is negligible. The Chebyshev distance between two allocation profiles is the maximum difference in the number of items received by any single agent. It captures the worst-case deviation experienced by any agent. The low Chebyshev distance of any two different χ in $\mathcal{S}$ shows the stability of the family of stable allocations.

Theorem 2. For $\chi, \chi' \in \mathcal{S}$, it holds that $|h_i - h'_i| \leq 1$ for each agent $i \in [n]$, where $(h_1, \dots, h_n) = \mathbf{p}(\chi)$ and $(h'_1, \dots, h'_n) = \mathbf{p}(\chi')$. In other words, the Chebyshev distance $D(\mathbf{p}(\chi), \mathbf{p}(\chi')) \leq 1$.

Proof. Suppose the opposite, that $h_i \geq h'_i + 2$. We shall prove that χ admits a narrowing transfer and hence is nonstable, which contradicts the assumption $\chi \in \mathcal{S}$.

We build a graph G with n vertices. If an item is allocated to j in χ and allocated to k in χ' , where $k \neq j$, build an arc from j to k . Note that G may have duplicate arcs. Clearly, χ becomes χ' after all the arcs (i.e., reallocations) are applied.

Decompose G into paths $P_1, \dots, P_b$ and a few cycles. Denote by s_j, t_j the starting and ending vertices of P_j , respectively. We assume that $t_j \neq s_k$ for $j \neq k$ as in Lemma 1. Moreover, assume that $s_1 = s_2 = i$ (due to $h_i \geq h'_i + 2$).

For $0 \leq j \leq b$, let $\chi^{(j)}$ be the allocation copied from χ but applied all the arcs (reallocations) in $P_1, \dots, P_j$. $\chi^{(0)} = \chi$.

Because $\mathbf{p}(\chi^{(b)}) = \mathbf{p}(\chi')$, we have $\text{LexiMin}(\chi^{(b)}) = \text{LexiMin}(\chi')$. We also know $\text{LexiMin}(\chi^{(0)}) = \text{LexiMin}(\chi)$ since both $\chi^{(0)}, \chi'$ are stable (Lemma 1). Together,

$\text{LexiMin}(\chi^{(0)}) = \text{LexiMin}(\chi^{(b)})$. It further implies that $\text{LexiMin}(\chi^{(0)}) = \text{LexiMin}(\chi^{(1)}) = \dots = \text{LexiMin}(\chi^{(b)})$. Otherwise, there exists j , such that $\text{LexiMin}(\chi^{(j)}) < \text{LexiMin}(\chi^{(j-1)})$, which means $\chi^{(j-1)}$ admits a narrowing transfer $s_j \rightarrow t_j$, implying that $\chi^{(0)}$ admits a narrowing transfer $s_j \rightarrow t_j$.

Since $\text{LexiMin}(\chi^{(0)}) = \text{LexiMin}(\chi^{(1)}) = \text{LexiMin}(\chi^{(2)})$, we know $s_1 \rightarrow t_1$, i.e., $i \rightarrow t_1$ (recall $s_1 = i$), is a swapping transfer of $\chi^{(0)}$, whereas $s_2 \rightarrow t_2$, i.e., $i \rightarrow t_2$ (recall $s_2 = i$), is a swapping transfer of $\chi^{(1)}$. It follows that $h_{t_2} = h_i - 2$; therefore, $\chi^{(0)} = \chi$ admits a narrowing transfer $i \rightarrow t_2$. $\square$

Remark 3. We will see in a later part of this paper (Theorem 5) that for the scenario of IND-SUB, the Chebyshev distance between different optimal allocations remains at most one. The proof of this generalized result is much more involved and is given in Section 5.

According to Theorem 2, the income h_i under different stable allocations χ does not differ too much for every agent i , so every solution χ seems to be acceptable for all of them. However, many questions regarding stable allocations remain to be settled. For example:

- Q₁. Are there common properties of all stable allocations?
- Q₂. Can we obtain the range of h_i over all stable allocations?
- Q₃. Is it possible to efficiently count the number of stable allocations?
- Q₄. Can we find out the profile (of some stable allocation) that optimizes a specific function of $h_1, \dots, h_n$?

In what follows, we introduce a construct, called “layer partition” of agents and items, which is independent of stable allocation χ . This construct elucidates the structure of $\mathcal{S}$ and, as we will demonstrate, provides a pathway to answering the four questions above.

First of all, we point out that each agent i falls into two cases: (1) its income h_i is always equal to some constant d for all $\chi \in \mathcal{S}$; or (2) its income h_i equals d for some $\chi \in \mathcal{S}$ and equals $d - 1$ for some other $\chi \in \mathcal{S}$, for some integer $d > 0$. This is a corollary of Theorem 2.

Definition 4 (Layer partition of agents and items). Denote by layer_d the set of agents i for which h_i always equal d (for $\chi \in \mathcal{S}$), and by layer_d^- the set of agents i for which h_i sometimes equal d and sometimes equal $d - 1$ (for $\chi \in \mathcal{S}$).

Take an arbitrary stable allocation $\chi \in \mathcal{S}$. Denote by Layer_d (and Layer_d^- , respectively) the set of items allocated to agent $i \in \text{layer}_d$ (and $i \in \text{layer}_d^-$, respectively) by χ .

The layer partition of agents and items consists [san] of the partition of agent $\text{layer}_0, \text{layer}_1, \text{layer}_1^-, \text{layer}_2^-, \dots$ and the partition of items $\text{Layer}_0, \text{Layer}_1, \text{Layer}_1^-, \text{Layer}_2^-, \dots$.

Lemma 2. Under each stable allocation $\chi \in \mathcal{S}$, the items from Layer_d (or Layer_d^- , respectively) are always allocated to the agents from layer_d (or layer_d^- , respectively).

Equivalently speaking, the set of items allocated to layer_d ($d \geq 0$) is invariant under every stable allocation. and the set of items allocated to layer_d^- (for $d > 0$) is invariant as well.

Shortly, items in any given layer can only be allocated to agents in that layer. To be clear, we regard $\text{layer}_d, \text{Layer}_d$ as in the same layer and $\text{layer}_d^-, \text{Layer}_d^-$ as in another layer.

3.2. Proof of Lemma 2 and the Explicit Construction of the Layer Partition

We prove Lemma 2 and show how to compute $\text{layer}_d, \text{layer}_d^-, \text{Layer}_d, \text{Layer}_d^-$ efficiently in this subsection. To this end, we introduce some additional notation first.

Fix any stable allocation $\chi \in \mathcal{S}$. Recall the transfers on χ (and the notation $\rightarrow$) in Definition 3. Those agents receiving d items under χ can be classified into three groups:

$$p_d := \{i \mid h_i = d \text{ and } \exists i \rightarrow j \text{ with } h_j = d - 1\}; \quad (1)$$

$$q_d := \{i \mid h_i = d \text{ and } \exists k \rightarrow i \text{ with } h_k = d + 1\}; \quad (2)$$

$$r_d := \{i \mid h_i = d, i \notin p_d, \text{ and } i \notin q_d\}. \quad (3)$$

See Figure 2a for an illustration of p_d, q_d and r_d . Observe that p_d is disjoint from q_d . Otherwise, there is $k \rightarrow j$ with $h_k = h_j + 2$, contradicting the fact that χ is stable. Denote $s_d = p_d \cup q_{d-1}$. Be aware that the quantity h_i , the groups p_d, q_d, r_d , and the set s_d all depend on the selected stable allocation χ .

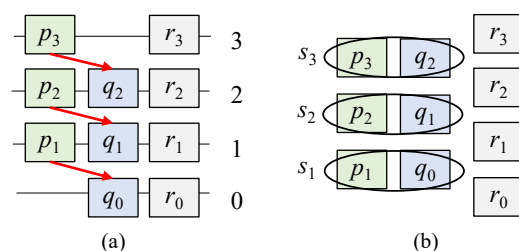


Figure 2. (a) Illustration of p_d, q_d, r_d . The arrows indicate that every agent in p_{d+1} has a transfer to some agent in q_d , and for each agent in q_d , there exists a transfer from some agent in p_{d+1} . (b) Illustration of s_d and r_d .

Denote by R_d and S_d the set of items allocated to r_d and s_d under χ , respectively.

Moreover, within this subsection, we rank all items and agents as follows:

- Elements in r_0, R_0 have rank 0; elements in s_1, S_1 have rank 1;
- Elements in r_1, R_1 have rank 2; elements in s_2, S_2 have rank 3, etc.

Proposition 1. In the aforementioned stable allocation χ ,

1. There is no transfer from s_d to any lower-ranked agents (namely, $r_{d-1}, s_{d-1}, \dots$);
2. There is no transfer from r_d to any lower-ranked agents (namely, $s_d, r_{d-1}, \dots$).

This easily follows from the definitions of p_d, q_d, r_d . Trivial proofs are omitted.

In the following, the rank refers to the rank defined above with respect to the fixed χ .

Proposition 2. In any stable allocation χ' , it holds that

1. A higher-ranked item cannot be allocated to a lower-ranked agent.
2. A lower-ranked item cannot be allocated to a higher-ranked agent.
3. Together, items in R_d are always assigned to r_d , and items in S_d are always assigned to s_d .

Proof. 1. An item o from S_d (or R_d , respectively) is not attractive to any lower-ranked agent, since otherwise χ admits a transfer from s_d (or r_d , respectively) to a lower-ranked agent (using just one reallocation of o), which contradicts Proposition 1. As a result, an item from S_d (or R_d , respectively) cannot be allocated to any lower-ranked agent in χ' .

2. We prove it by induction. In this proof, for the simplicity of presentation, assume $\max\{h_i\} = 3$; see Figure 2. First, the items ranked lower than r_3 cannot be allocated to r_3 in χ' . Otherwise, as the items in R_3 must be allocated to r_3 (by the analysis above), some agent in r_3 receives more than 3 items by the pigeonhole principle, which implies that $\mathbf{p}(\chi')$ is non-optimal under LexiMax, and therefore χ' is not stable (Theorem 1). Similarly, the items ranked lower than s_3 cannot be allocated to s_3 in χ' ; otherwise, $\mathbf{p}(\chi')$ contains more agents receiving 3 items than $\mathbf{p}(\chi)$, which is impossible, and so on and so forth. $\square$

Proposition 3. For $i \in r_d$, it holds that h_i is always equal to d for all $\chi \in \mathcal{S}$; and for $i \in s_d$, it holds that h_i can be d or $d - 1$ under different $\chi \in \mathcal{S}$. It follows that $\text{layer}_d = r_d$ and $\text{layer}_d^- = s_d$.

Proof. According to Claim 3 of Proposition 2, in every stable allocation, agents r_d will receive and only receive R_d . Note that $|R_d| = d|r_d|$. Therefore, in every stable allocation, each agent from r_d receives exactly d items (otherwise, there must be one agent receiving more than d and one agent receiving less than d , which is worse than an equal distribution for the criteria LexiMin and hence nonstable). Hence, h_i always equals d for $i \in r_d$.

For each $i \in p_d$, we know $h_i = d$, and h_i can be reduced to $d - 1$ (due to (1)) in another stable allocation. For each $i \in q_{d-1}$, we know $h_i = d - 1$, and h_i can be increased to d (due to (2)) in another stable allocation. Therefore h_i can be $d - 1$ or d for $i \in p_d \cup q_{d-1} = s_d$. $\square$

Lemma 2 simply follows from all the analysis above. In particular, combining the last two propositions, we obtain that $\text{Layer}_d = R_d$ and $\text{Layer}_d^- = S_d$.

3.3. Some Applications of the Layer Partition—Answering $Q_1, \dots, Q_4$

With the layer partition of agents and items, we can promptly answer Q_1 to Q_4 :

Answer of Q_1 . They all allocate items layer by layer, in which the layers are invariant with respect to the chosen allocation.

Answer of Q_2 . For $i \in \text{layer}_d$, the range of h_i is $[d, d]$. For $i \in \text{layer}_d^-$, the range of h_i is $[d - 1, d]$. Computing the layers reduces to computing r_d, s_d for any fixed χ , which is easy. Hence we can compute the ranges of h_i s efficiently.

Alternative methods for computing the ranges have to use network flow and are far more complicated.

Answer of Q_3 . Counting stable allocations reduces to counting stable allocations within each layer (and then applying the rule of product).

However, this counting problem is #P-hard, as the problem of counting stable allocation in layer_1 can be reduced to counting perfect matchings, which is already #P-hard.

Answer of Q_4 . Finding a profile (of a stable allocation) that minimizes a linear function of $h_1, \dots, h_n$, e.g., $\sum c_i h_i$, is easy. Modify the network used in the proof of Claim 2 of Theorem 1 as follows: Among the edges from v_j to t , change the cost of the k th one to be $(k - 1) \times A + c_i$, where A is a large enough constant. Then, the minimum-cost flow still has the minimum Congestion, and it optimizes $\sum c_i h_i$. For non-linear functions, it is more difficult. Yet the layer partition still helps break down the task.

Remark 4. Halpern et al. [9] implicitly found a structure similar to our layer structures. Specifically, (in the proof of their Theorem 4), given a fractional MNW allocation, they partition the agents into subsets according to the floor of valuations (so in a certain set, the valuation range of each agent is $[x, x + 1)$ for some integer x). They imply that the agents in each set must be fully allocated to a certain subset of items for any fractional MNW allocation, and the partition and correspondence between the subsets of agents and the subsets of items also hold for deterministic MNW allocations. We discover our layer structures independently. Conceptually, our layers are defined not merely by valuation intervals but by partitions of the agents who receive the same number of items according to the transferring relation between agents. Theoretically, the presence of the layer partition of IND was directly confirmed in the scenario of IND, without the assistance of the DIV case. And so is the computation of the layer partition of IND. Observationally, our layer structures are more specific, which can be briefly explained through the valuation range of each agent in a certain layer: for the scenario of DIV, compared to their range of $[x, x + 1)$ for some integer x , the range in our layer structures is some fixed rational number, and for the scenario of IND, compared to their range of

$[x, x + 1]$ for some integer x , the range in our layer structures is $[x, x + 1]$ for some integer x or some fixed integer (the range of $[x, x]$).

4. Divisible Items and Agents with 0/1-Add Valuations

This section discusses the scenario of DIV, i.e., the case of divisible items, in which we are allowed to allocate a part of an item to an agent (and different parts to different agents, perhaps). Note that the 0/1-sub valuations cannot easily match with divisible items. Therefore we restrict ourselves to 0/1-add valuations in this section.

Definition 5 (Stable allocations in the scenario of DIV). Let χ be an allocation of divisible items. For each part of item o allocated to agent i that can be reallocated to another agent i' (i.e., $v_i(\{o\}) = 1$), build an edge (i, i') . Moreover, if there is a simple path $(i_1, \dots, i_k)$ ($k \geq 2$) along such edges, we state that χ admits a transfer from $i_1 = u$ to $i_k = v$, denoted by $u \rightarrow v$. It consists of $k - 1$ reallocations with $\Delta > 0$ fraction of items along the path, after which h_u decreases by Δ and h_v increases by Δ .

A narrowing transfer refers to a transfer from u to v with $h_u - \Delta \geq h_v + \Delta$. A widening transfer refers to a transfer from u to v with $h_u \leq h_v$. An Allocation χ is called nonstable if it admits some narrowing transfer and is stable otherwise.

Denote by $\mathcal{S}^*$ the set of stable allocations in the DIV scenario.

Lemma 3. Stable allocations are optimal under LexiMin. (As a corollary, their profiles are equivalent up to permutation.)

Proof. Assume χ is non-optimal under LexiMin. We shall prove that χ is nonstable, i.e., it admits a narrowing transfer.

First, take an allocation χ^* that is optimal under LexiMin.

We build a graph G with n vertices. Be aware that according to χ and χ^* , each item i can be divided into several pieces $i_1, \dots, i_p$ (with a total size of one), so that each piece is given to a certain agent (denoted by j) in χ and given to a certain agent (denoted by k) in χ^* . If $j \neq k$, build an arc from j to k , with weight equal to the size of this piece. Clearly, an arc represents a reallocation (of one piece) on χ , and χ becomes χ^* after all the arcs (i.e., reallocations) are applied.

We decompose graph G into several cycles $C_1, \dots, C_a$ and paths $P_1, \dots, P_b$, where the edges in any cycle or path have the same weight, and where $t_j \neq s_k$ for $j \neq k$, where s_i, t_i denote the starting and ending vertices of P_i , respectively. Such a decomposition exists under an appropriate division of items.

For $0 \leq i \leq b$, let $\chi^{(i)}$ be the allocation copied from χ but applied all the arcs (reallocations) in $P_1, \dots, P_i$. $\chi^{(0)} = \chi$.

Be aware that $\chi^{(b)}$ becomes χ^* after applying the arcs in $C_1, \dots, C_a$. We obtain that $\text{LexiMin}(\chi^{(b)}) = \text{LexiMin}(\chi^*)$. Further since that $\text{LexiMin}(\chi^*) < \text{LexiMin}(\chi)$, there exists i ($1 \leq i \leq b$), such that $\text{LexiMin}(\chi^{(i)}) < \text{LexiMin}(\chi^{(i-1)})$.

It follows that in $\chi^{(i-1)}$, we have $h_s > h_t$ (where s, t denote s_i, t_i respectively, for short). It further follows that in $\chi^{(0)} = \chi$, we also have $h_s > h_t$, since h_s never increases and h_t never decreases in the sequence $\chi^{(0)}, \dots, \chi^{(i-1)}$. Consequently, χ admits a narrowing transfer (from s to t). $\square$

Theorem 3. 1. For any SPD criterion, the optimums coincide with $\mathcal{S}^*$.
2. We can find a stable allocation in polynomial time.

Just like we prove Claim 1 of Theorem 1 using Lemma 1, we can prove Claim 1 of Theorem 3 using Lemma 3 (the proof is omitted).

To find a stable allocation in polynomial time, we can use an approach based on linear programming (LP), which is shown in Section 4.2 (thus we prove Claim 2 of Theorem 3). Alternatively, we find a purely combinatorial approach (which is more efficient and more interesting) for finding such an allocation. It utilizes the layer partition with several nontrivial ideas. See details in Section 4.1.

Theorem 4. For $\chi, \chi' \in \mathcal{S}^*$, it holds that $\mathbf{p}(\chi) = \mathbf{p}(\chi')$; namely, the Chebyshev distance $D(\mathbf{p}(\chi), \mathbf{p}(\chi')) = 0$.

A proof of Theorem 4 is related to the LP approach mentioned above and is given in Section 4.2. An alternative proof is similar to the proof of Theorem 2 and is omitted.

4.1. Layer Partition of Agents and Items for Divisible Items and a Combinatorial Algorithm for Finding a Stable Allocation

In the following, we extend the layer partition given in Section 3.1 to the divisible case and then present the aforementioned combinatorial algorithm.

According to Theorem 4, profile $\mathbf{p}(\chi)$ is unique for $\chi \in \mathcal{S}^*$. In other words, the income h_i of each agent i is independent of χ , as long as χ is stable.

For each real number d , denote by layer_d^* the set of agents that always receive d items no matter in which stable allocation; formally, $\text{layer}_d^* = \{i \mid h_i = d\}$.

Lemma 4. The set of items allocated to layer_d^* is invariant for $\chi \in \mathcal{S}^*$. Moreover, this set (denoted by Layer_d^* henceforth) consists of complete items only.

Proof. Fix χ . Let L_d be the items allocated to layer_d^* under χ . An item in $L_{d'}$ ($d' > d$) cannot be allocated to layer_d^* ; otherwise, there is a narrowing transfer in χ . An item in $L_{d'}$ ($d' < d$) cannot be allocated to layer_d^* ; otherwise, the allocation is not LexiMax optimal (can be proved by induction as in the proof of Proposition 2). Therefore, those items in L_d can only be allocated to layer_d^* (even for other stable allocations). We thus obtain the first part of this lemma.

In any stable allocation, an item cannot be allocated to different layers. Otherwise there is clearly a simple narrowing transfer. $\square$

Lemma 4 implies a layer partition of items and agents, where items Layer_d^* and agents layer_d^* are in the same layer.

Lemma 5. A stable allocation γ for the divisible case can be obtained by optimally reallocating items Layer_d^- (regarded as divisible items) to agents layer_d^- and items Layer_d^+ (regarded as indivisible items) to layer_d^+ for each d .

Proof. For each d , let γ_d^- be an optimal allocation of items Layer_d^- (regarded as divisible items) to agents layer_d^- . Moreover, let γ_d^+ be an optimal allocation of items Layer_d^+ (regarded as indivisible items) to layer_d^+ so that each agent in layer_d^+ receives d items. Combine γ_d, γ_d^- ($d \geq 0$) in all layers to obtain an overall allocation γ . We claim that $\gamma \in \mathcal{S}^*$. The proof is as follows.

Those items in the higher layer cannot be given to agents in the lower layer—there are no such edges as shown in the proof of Proposition 2 (this also holds for the divisible case). Hence there is no narrowing transfer between layers in γ . Also, there is no narrowing transfer within any layer of γ by the construction of γ . Together, γ admits no narrowing transfer and hence is stable. So, $\gamma \in \mathcal{S}^*$. $\square$

Lemma 6. For any $\chi \in \mathcal{S}^*$, it holds that

1. $h_i = d$ for each $i \in \text{layer}_d$.
2. $h_i \in [d - 1, d]$ for $i \in \text{layer}_d^-$.

Proof. Denote $(g_1, \dots, g_n) = \mathbf{p}(\gamma)$ and $(h_1, \dots, h_n) = \mathbf{p}(\chi)$.

Combining (1)–(3) below, we immediately obtain the lemma:

- (1) For each $i \in [n]$, it holds that $h_i = g_i$ (apply Theorem 4 with $\gamma, \chi \in \mathcal{S}^*$).
- (2) For each $i \in \text{layer}_d$, it holds that $g_i = d$ (trivial).
- (3) For each $i \in \text{layer}_d^-$, it holds that $g_i \in [d - 1, d]$. (Proof: Since γ_d^- is (LexiMin) optimal, $g_i \geq d - 1$. Since γ_d^- is also (LexiMax) optimal, $g_i \leq d$.)

□

Lemma 7. Given $\chi \in \mathcal{S}^*$, for two different layers layer_d^* and $\text{layer}_{d'}^*$ of χ , we have

$$|d - d'| > \frac{1}{n^2}.$$

Proof. By Lemma 5, we obtain that

$$d = \frac{|\text{Layer}_d^*|}{|\text{layer}_d^*|}, \quad d' = \frac{|\text{Layer}_{d'}^*|}{|\text{layer}_{d'}^*|}.$$

According to Lemma 4, we have that $|\text{Layer}_d^*|$ and $|\text{Layer}_{d'}^*|$ are integers and $|\text{layer}_d^*| + |\text{layer}_{d'}^*| \leq n$.

Thus, $||\text{Layer}_d^*|||\text{layer}_{d'}^*| - |\text{Layer}_{d'}^*|||\text{layer}_d^*|| \geq 1$ and $|\text{layer}_d^*|||\text{layer}_{d'}^*| < n^2$.

Therefore,

$$|d - d'| = \left| \frac{|\text{Layer}_d^*|||\text{layer}_{d'}^*| - |\text{Layer}_{d'}^*|||\text{layer}_d^*||}{|\text{layer}_d^*|||\text{layer}_{d'}^*|} \right| > \frac{1}{n^2}.$$

□

The last three lemmas are crucial to our combinatorial algorithm for DIV.

For the divisible case, we divide each item into n^2 identical (unbreakable) pieces with size $\frac{1}{n^2}$, and solve this indivisible case (with $n^2 m$ indivisible pieces) using the algorithm in Claim 2 of Theorem 1.

We obtain an allocation with several layers; each layer's index d is a real number—a multiple of $\frac{1}{n^2}$. For agent $j \in \text{layer}_d$, we have $h_j = d$, and for $i \in \text{layer}_d^-$, we have $h_i \in [d - \frac{1}{n^2}, d]$.

According to Lemma 6, for the divisible case, we have

$$h_i \in [d - \frac{1}{n^2}, d] \text{ for } i \in \text{layer}_d^-.$$

As a result, $i \in \text{layer}_{d'}^*$, for some $d' \in [d - \frac{1}{n^2}, d]$.

By Lemma 7, there can only be one such d' in the range of $[d - \frac{1}{n^2}, d]$ for $i \in \text{layer}_d^-$. By Lemma 5, we can obtain an optimal allocation for the divisible case just by reallocating items within each layer, and we can calculate that $d' = \frac{|\text{Layer}_d^-|}{|\text{layer}_d^-|}$. See Algorithm 2 for details.

Algorithm 2 Finding stable allocation for DIV via reduction to IND

Input: m items $[m]$, n agents $[n]$, binary valuation matrix B of size $n \times m$, where $B(i, j)$ denotes the valuation of item i for agent j .

Output: A stable allocation χ^* for divisible items.

- 1: Divide each item $j \in [m]$ into n^2 identical pieces of size $\frac{1}{n^2}$; let $\mathcal{P}$ denote the set of all these pieces.
- 2: Expand B to B' , such that for each piece o of item j , we have $B'(i, o) = B(i, j)$.
- 3: $\gamma \leftarrow \text{STABLEALLOCATIONIND}(\mathcal{P}, [n], B')$ {Algorithm 1}.
- 4: Compute layer partition of γ : agents $\text{layer}_d, \text{layer}_d^-$ and items $\text{Layer}_d, \text{Layer}_d^-$.
{With answer of Q_2 }
- 5: **for** each layer d of γ with agents layer_d and items Layer_d **do**
- 6: Allocate items in Layer_d integrally to agents in layer_d , such that each gets d value in χ^*
- 7: **end for**
- 8: **for** each layer d of γ with agents layer_d^- and items Layer_d^- **do**
- 9: Allocate items in Layer_d^- fractionally to agents in layer_d^- , such that each agent receives exactly $|\text{Layer}_d^-|/|\text{layer}_d^-|$ value in χ^*
- 10: **end for**
- 11: **return** χ^* .

Remark 5. The parameter $\frac{1}{n^2}$ is a sufficient choice. By Lemma 6, each layer_d of γ corresponds to a layer_d^* of χ^* , and each layer_d^- corresponds to all $\text{layer}_{d'}^*$ with $d' \in [d - \frac{1}{n^2}, d]$. By Lemma 7, two layers d, d' in the divisible case must be $\frac{1}{n^2}$ away from each other, so there can only be one such d' in the range of $[d - \frac{1}{n^2}, d]$.

As a result, each layer in the indivisible reduction corresponds to only one layer in the original divisible case. By Lemma 5, we can obtain an optimal allocation for the divisible case just by reallocating items within each layer, and we can calculate the value for each layer since the number of layers (in the divisible case) is guaranteed to be one.

In addition, a subdivision into $O(n^2)$ pieces is inevitable. For example, suppose we have $n = 2k + 1$ agents and $2k - 1$ items. The first $k + 1$ agents can only receive the first k items (arbitrarily), and the remaining k agents can only receive the remaining $k - 1$ items (arbitrarily). The optimal result of this case is obviously allocating items equally if possible. In the end, the first $k + 1$ agents become a layer getting the k items (evenly distributed), and the rest become the other layer getting the $k - 1$ items (evenly distributed). The difference between these two layers is $\frac{k}{k+1} - \frac{k-1}{k} = \frac{1}{k^2+k}$. So the denominator is $O(n^2)$ for $n = 2k + 1 = O(n)$, indicating that we need $O(n^2)$ pieces to indicate such a small difference.

The obtained algorithm deals with mn^2 items and n agents. Therefore, it has a time complexity of $O(m^2n^5)$, which is still better than the linear programming procedure (explained below) with time complexity of $O(m^{3.5}n^{5.5})$ with $O(mn)$ variables and $O(n^2)$ linear programming problems in the worst case.

Example 2. A numerical illustration of Algorithm 2.

Consider an instance with $n = 5$ agents and $m = 3$ divisible items, where the binary valuation matrix B is:

$$B = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 \end{pmatrix}.$$

Our combinatorial algorithm proceeds as follows:

Step 1: Split each item into $n^2 = 25$ identical, indivisible pieces. We now have $3 \times 25 = 75$ pieces in total.

Step 2: Compute a stable allocation for these 75 indivisible pieces using Algorithm 1. One possible optimal allocation yields the following piece counts:

- Agent 1 receives 12 pieces from item 1.
- Agent 2 receives 13 pieces from item 1.
- Agent 3 receives 16 pieces from item 2.
- Agent 4 receives 9 and 8 pieces from items 2 and 3, respectively.
- Agent 5 receives 17 pieces from item 3.

The final profile (piece counts) is $(h_1, h_2, h_3, h_4, h_5) = (12, 13, 16, 17, 17)$. Converting back to the original item scale (dividing by 25), we obtain the profile: $(\frac{12}{25}, \frac{13}{25}, \frac{16}{25}, \frac{17}{25}, \frac{17}{25})$.

Step 3: Compute the layer partition.

- Agents 1 and 2 are in layer_d^- , and item 1 is in Layer_d^- , where $d = \frac{13}{25}$.
- Agents 3, 4 and 5 are in $\text{layer}_{d'}^-$, and items 2 and 3 are in $\text{Layer}_{d'}^-$, where $d' = \frac{17}{25}$.

Step 4: Recompose the fractional allocation. Agents 1 and 2 each receives $\frac{1}{2}$ item, and agents 3, 4 and 5 each receives $\frac{2}{3}$ item. The profile of stable allocation is $(h_1, h_2, h_3, h_4, h_5) = (\frac{1}{2}, \frac{1}{2}, \frac{2}{3}, \frac{2}{3}, \frac{2}{3})$.

4.2. Find a Stable Allocation for Divisible Items Using LP and a Proof of Theorem 4

This subsection provides an alternative approach based on linear programming for finding a stable allocation for divisible items. This approach is more straightforward, but it is not combinatorial.

Applying Claim 1 of Theorem 3, finding a stable allocation reduces to finding an allocation with the minimum LexiMin.

First, we prove the claim that the latter further reduces to computing the “fair multi-flow” [11] in the network below:

See Figure 3. There are n source nodes $s^1, \dots, s^n$ in the first layer S . The second layer R consists of only one node r as a relay. The third layer I has m nodes, standing for items. The last layer T has n sink nodes $t^1, \dots, t^n$. For each agent d , there is an arc (s^d, r) of unlimited capacity. For every item i in I , there is an arc (r, i) of capacity one. If item i can be allocated to agent d , an arc (i, t^d) is added with capacity one.

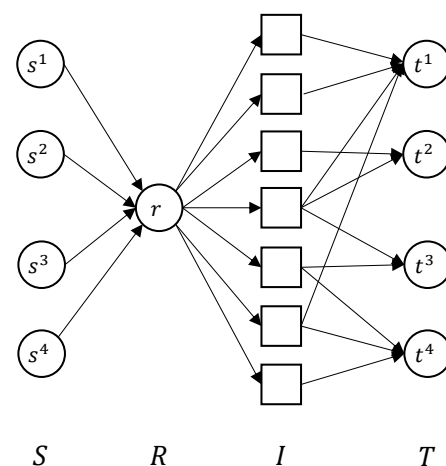


Figure 3. Reduction to fair multi-flow problem. The reduction constructs a directed acyclic network $G = (V, E)$ with four layers. The key insight is to encode the LexiMin objective as a fair multi-flow problem, where the amount of flow received by each sink directly corresponds to the valuation of the respective agent.

In the aforementioned fair multi-flow problem, we shall send a flow f_d from s^d to t^d for each d . The sum of the flows on each edge cannot exceed the capacity, and the n amount of flows $|f_1|, \dots, |f_n|$ as a vector should have the lowest LexiMin. Clearly, the solution to

this problem corresponds to the optimal allocation under LexiMin (items are fully allocated as the multi-flow is LexiMin); therefore, the claim holds.

Then, recall the result of [11], which states that the fair multi-flow problem can be reduced to a polynomial number of linear programming problems and can thus be solved in polynomial time.

Proof of Theorem 4. Recall finding a stable allocation by computing a fair multi-flow; the flow vector $(|f_1|, \dots, |f_n|)$ is unique for the multi-flow problem, according to [11]. Therefore the profile of optimal allocation is also unique. $\square$

5. Indivisible Items and Agents with 0/1-Sub Valuations

We now move on to the scenario of IND-SUB.

The 0/1-sub function is closely related to the matroid theory [19]. A matroid is a pair $(E, \mathcal{I})$, where E is a finite set (called the *ground set*) and $\mathcal{I}$ is a family of subsets of E (called the *independent sets*).

The independent sets satisfy the following three axioms:

- (I1) $\emptyset \in \mathcal{I}$;
- (I2) If $Y \in \mathcal{I}$ and $X \subseteq Y$, then $X \in \mathcal{I}$;
- (I3) If $X, Y \in \mathcal{I}$ and $|X| < |Y|$, then there exists $y \in Y \setminus X$, such that $X \cup \{y\} \in \mathcal{I}$.

If an agent has a 0/1-sub valuation on items, then the set of *clean* bundles forms the set of independent sets of a matroid (proved in Benabbou et al. [7]). Benabbou et al. [7] further showed that under 0/1-sub valuations, the set of max-USW allocations optimal under any SPD criterion is consistent with the set of LexiMin allocations. This result generalizes Claim 1 of Theorem 1 (for the 0,1-add valuations). In addition, Babaioff et al. [10] showed that under 0/1-sub valuations, the allocation that optimizes $\text{NSW}^\top$ can be found in polynomial time.

To the best of our knowledge, however, it is not known to what extent can $\mathbf{p}(\chi), \mathbf{p}(\chi')$ differ for different optimal allocations χ, χ' . We answer this question by the following theorem.

Let $\mathcal{SS}$ denote the set of (max-USW and clean) allocations that are optimal under any SPD criterion.

Theorem 5. For $\chi, \chi' \in \mathcal{SS}$, it holds that $|h_i - h'_i| \leq 1$ for each agent $i \in [n]$, where $(h_1, \dots, h_n) = \mathbf{p}(\chi)$ and $(h'_1, \dots, h'_n) = \mathbf{p}(\chi')$. In other words, the Chebyshev distance is $D(\mathbf{p}(\chi), \mathbf{p}(\chi')) \leq 1$.

Proof. Assume (i) $\chi, \chi' \in \mathcal{SS}$ and (ii) $D(\mathbf{p}(\chi), \mathbf{p}(\chi')) \geq 2$. If there are multiple such pairs of allocations, take the pair (χ, χ') with a minimum symmetric difference of $\sum_{i \in [n]} |\chi_i \triangle \chi'_i|$.

Without loss of generality, assume that

$$h_1 \leq h_2 \leq \dots \leq h_n \text{ and } h_q \geq h'_q + 2.$$

Assume that $h'_{j_1} \leq h'_{j_2} \leq \dots \leq h'_{j_n}$.

Notice that $\mathbf{p}(\chi) \neq \mathbf{p}(\chi')$. Take the minimum index i satisfying $h_i \neq h'_i$. Clearly, for all $k \in [i-1]$ we have

$$h_k = h'_k. \quad (4)$$

In fact, for i , it must hold that

$$h_i < h'_i. \quad (5)$$

Recall that the smaller $\text{LexiMin}(\mathbf{p})$, the better the vector $\mathbf{p}$ under LexiMin. Indeed, if $h_i > h'_i$, then together with (4),

$$\text{LexiMin}((h_1, \dots, h_i)) > \text{LexiMin}((h'_1, \dots, h'_i)).$$

Moreover,

$$\text{LexiMin}((h'_1, \dots, h'_i)) \geq \text{LexiMin}((h'_{j_1}, \dots, h'_{j_i})),$$

since $h'_{j_1}, \dots, h'_{j_i}$ are the smallest i elements in $\mathbf{p}(\chi')$. Together, we have $\text{LexiMin}((h_1, \dots, h_i)) > \text{LexiMin}((h'_{j_1}, \dots, h'_{j_i}))$. By comparing the smallest i elements of $\mathbf{p}(\chi)$ and $\mathbf{p}(\chi')$, we see $\text{LexiMin}(\mathbf{p}(\chi)) > \text{LexiMin}(\mathbf{p}(\chi'))$, contradicting the assumption that χ and χ' are both LexiMin optimal.

By the definition of i and the assumption $h_q \geq h'_q + 2$, it is easy to see $q > i$ and hence $h_q \geq h_i$. Further, we claim

$$h_q \geq h_i + 2. \quad (6)$$

It reduces to prove $h'_q \geq h_i$. Since χ and χ' are both LexiMin optimal, the two multisets $\{h_1, \dots, h_n\}$ and $\{h'_1, \dots, h'_n\}$ are equivalent. By (4), for all $k \in [i-1]$, $h_k = h'_k$. Together, the multiset $\{h_i, \dots, h_n\}$ equals $\{h'_i, \dots, h'_n\}$. Further by the assumption $h_1 \leq \dots \leq h_n$, the elements in $\{h'_i, \dots, h'_n\}$ are not smaller than h_i . Recall $q > i$, we have $h'_q \geq h_i$.

Recall that the family of clean bundles $\mathcal{I}_j = \{S \subseteq [m] \mid v_j(S) = |S|\}$ for $j \in [n]$ forms a family of independent sets of a matroid [7]. By (I3) of the independent-set matroid axioms and inequality (5), there exists an item $o_1 \in \chi'_i \setminus \chi_i$ making $v_i(\chi_i \cup \{o_1\}) = v_i(\chi_i) + 1 = h_i + 1$. And o_1 is allocated to some agent $i_1 \neq i$ under allocation χ . Otherwise, o_1 is not allocated to anyone, and can be allocated to agent i , violating that χ is max-USW. Consider the following three cases:

1. Suppose $h_{i_1} \geq h_i + 2$. Then transferring o_1 from i_1 to i in χ decreases $\text{LexiMin}(\mathbf{p}(\chi))$, contradicting that χ is LexiMin optimal.
2. Suppose $h_{i_1} = h_i + 1$. We note that $i_1 \neq q$ since $h_q \geq h_i + 2$ (inequality (6)). If we transfer o_1 from i_1 to i in χ , $\text{LexiMin}(\mathbf{p}(\chi))$ and h_q are unchanged, which means (χ, χ') still satisfies the two conditions (i) $\chi, \chi' \in \mathcal{SS}$ and (ii) $D(\mathbf{p}(\chi), \mathbf{p}(\chi')) \geq 2$, but the $\sum_{i \in [n]} |\chi_i \triangle \chi'_i|$ decreases, a contradiction.
3. Suppose $h_{i_1} \leq h_i$.
We first show $h_{i_1} \leq h'_{i_1}$. This clearly holds by (4) if $i_1 < i$.
When $i_1 > i$, since $h_1 \leq \dots \leq h_n$, we have $h_{i_1} \geq h_i$. Together with the assumption $h_{i_1} \leq h_i$ in this case, we have $h_{i_1} = h_i$.
Suppose to the oppose that $h_{i_1} > h'_{i_1}$, which means $h_i > h'_{i_1}$. Applying (4),

$$\text{LexiMin}((h_1, \dots, h_{i-1}, h_i)) > \text{LexiMin}((h'_1, \dots, h'_{i-1}, h'_{i_1})).$$

Moreover,

$$\text{LexiMin}((h'_1, \dots, h'_{i-1}, h'_{i_1})) \geq \text{LexiMin}((h'_{j_1}, \dots, h'_{j_i})),$$

since $h'_{j_1}, \dots, h'_{j_i}$ are the smallest i elements in $\mathbf{p}(\chi')$. Together, $\text{LexiMin}((h_1, \dots, h_i)) > \text{LexiMin}((h'_{j_1}, \dots, h'_{j_i}))$, by comparing the smallest i elements of $\mathbf{p}(\chi)$ and $\mathbf{p}(\chi')$, yielding $\text{LexiMin}(\mathbf{p}(\chi)) > \text{LexiMin}(\mathbf{p}(\chi'))$, a contradiction.

With $h_{i_1} \leq h'_{i_1}$ (i.e., $v_{i_1}(\chi_{i_1}) \leq v_{i_1}(\chi'_{i_1})$) in hand, since χ_{i_1} is clean, we have $v_{i_1}(\chi_{i_1} \setminus \{o_1\}) < v_{i_1}(\chi'_{i_1})$. Further since $\chi_{i_1} \setminus \{o_1\}$ and χ'_{i_1} are clean (i.e., independent sets of a matroid), there exists an item $o_2 \in \chi'_{i_1} \setminus (\chi_{i_1} \setminus \{o_1\})$, such that $v_{i_1}(\chi_{i_1} \setminus \{o_1\} \cup \{o_2\}) = v_{i_1}(\chi_{i_1}) = h_{i_1}$. And o_2 is allocated to some agent $i_2 \neq i_1$ under χ , as otherwise o_2 is not allocated to anyone, and we can transfer o_1 from i_1 to i and allo-

cate o_2 to i_1 in χ , violating that χ is max-USW. We note that $\chi'_{i_1} \setminus (\chi_{i_1} \setminus \{o_1\}) = \chi'_{i_1} \setminus \chi_{i_1}$ and $o_2 \neq o_1$ because $o_1 \notin \chi'_{i_1}$ (recall that $o_1 \in \chi'_i \setminus \chi_i$) and $o_2 \in \chi'_{i_1}$.

Repeating the same argument and letting $i_0 = i$, we obtain a sequence of items and agents $(i_0, o_1, i_1, \dots, o_t, i_t)$. Let $\chi^{(k)}$ denote the allocation transferring o_l from i_l to i_{l-1} under χ for all $l \in [k]$. The sequence of items and agents satisfies $o_k \in \chi'_{i_{k-1}} \setminus \chi^{(k-1)}_{i_{k-1}}$.

See Figure 4 for an illustration of the sequence. We note that the same item o_l does not appear again, since for $k \geq l$, we have $o_l \in \chi^{(k)}_{i_{l-1}}$ (as a result, $o_l \notin \chi'_{i_{k-1}} \setminus \chi^{(k)}_{i_{l-1}}$). Thus, the sequence must terminate when we reach agent i_t with $h_{i_t} \geq h_i + 2$ or $h_{i_t} = h_i + 1$ for the first time. If $h_{i_t} \geq h_i + 2$, after transferring items along the sequence, we have that χ is not LexiMin optimal, a contradiction. If $h_{i_t} = h_i + 1$, we note that agent q (recall that $h_q \geq h'_q + 2$) is not in the sequence since $h_q \geq h_i + 2$ (inequality (6)). After transferring items along the sequence, LexiMin($\mathbf{p}(\chi)$) and h_q are unchanged, which means (χ, χ') still satisfies the two conditions (i) $\chi, \chi' \in \mathcal{SS}$ and (ii) $D(\mathbf{p}(\chi), \mathbf{p}(\chi')) \geq 2$, but $\sum_{i \in [n]} |\chi_i \Delta \chi'_i|$ decreases, a contradiction.

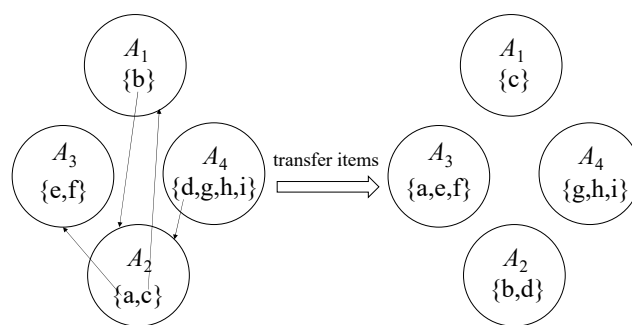


Figure 4. Illustration of transferring items along the sequence. Some items are denoted by $a, \dots, g$. Some agents and their bundles are denoted by $1, \dots, 4$ and $A_1, \dots, A_4$ respectively. In this example, the sequence $(i_0, o_1, i_1, \dots, o_t, i_t)$ is $(3, a, 2, b, 1, c, 2, d, 4)$.

To sum up, suppose the opposite of Theorem 5 that there are some pairs of allocations (χ, χ') that meet (i) $\chi, \chi' \in \mathcal{SS}$ and (ii) $D(\mathbf{p}(\chi), \mathbf{p}(\chi')) \geq 2$, then it leads to some contradiction. Thus, for $\chi, \chi' \in \mathcal{SS}$, it holds that $D(\mathbf{p}(\chi), \mathbf{p}(\chi')) \leq 1$. $\square$

Remark 6. Our proof of Theorem 5 is similar to that of Lemma 3.12 in [7], which is the 0/1-sub valuation version of Lemma 1 (implying an allocation non-optimal under LexiMin is non-optimal under any SPD criterion). There is a minor flaw in their proof that we have modified in our proof of Theorem 5.

In the proof of Lemma 3.12 in [7], for the sequence of items and agents $(i_0, o_1, i_1, \dots, o_t, i_t)$, they claim that no same **agent** appears twice by proof, beginning from arguing that if the same **agent** appears again, the allocation χ is still clean after transferring items along the cycle—a subsequence of the sequence with a common agent as the starting and ending element. However, the above claim and proof are incorrect. See the example in the left figure of Figure 4, where the sequence is $(3, a, 2, b, 1, c, 2, d, 4)$ and agent 2 appears twice. After transferring items along the cycle $(2, b, 1, c, 2)$ (which is equivalent to swapping items b and c between agents 2 and 1), the allocation may not be clean if $\{a, b\}$ is not a clean bundle of agent 2. Indeed, the family of clean bundles of agent 2 may be $\mathcal{I}_2 = \{\{a, c\}, \{a, d\}, \{b, c\}, \{b, d\}, \{a\}, \{b\}, \{c\}, \{d\}\}$, such that $\{a, b\} \notin \mathcal{I}_2$.

We note that Lemma 3.12 in [7] still holds, and the minor flaw in its proof in [7] can be corrected by arguing that no same **item** appears twice, like our proof of Theorem 5.

The layer structures of the IND and DIV scenarios admit the property that is within each layer; the number of items allocated to any two agents differs by at most a fixed and small constant that is independent of a specific instance—specifically, by at most *one* in IND

and exactly *zero* in DIV. This uniformity is enabled by the additive nature of valuations, which allows items to be freely transferred between agents without affecting their marginal gains on other items.

In the IND-SUB scenario, however, this regular layer structure breaks down. While the overall profiles of different optimal allocations remain close (Chebyshev distance ≤ 1 , as shown in Theorem 5), the underlying combinatorial rigidity imposed by submodular (matroid) constraints prevents the formation of stable, invariant layers, where allocations differ only by a bounded constant. A simple counterexample is an instance with two agents and ten items, with submodular valuations defined as: agents 1 and 2, with the value of any subset S of items respectively as $\min(x, |S|)$ and $\min(10 - x, |S|)$, where x is a parameter of the instance. The ten items can be freely allocated to the two agents, and the allocation is stable as long as agent 1 receives x items. Thus, the two agents are in the same layer. However, the difference in the number of items that these two agents received changes as the instance changes, rather than being a fixed constant of the whole IND-SUB scenario.

6. Summary

For the binary valuations, it is known that the set of optimums has nothing to do with the exact criterion, as long as the criterion is SPD and symmetric. A complete proof of this result was scattered in the literature and is provided in this paper. Furthermore, this paper discusses the consistency among the set of SPD optimal allocations, that is, their profiles are close to each other under Chebyshev distance, which holds for 0/1-sub valuations. The proof is nontrivial.

For the 0/1-add valuations, we introduce a layer partition among items and agents. Together with the idea of dividing items into n^2 identical pieces, we obtain the first combinatorial algorithm for computing an optimal allocation (i.e., a stable allocation) for the divisible items, which has a lower complexity than the LP approach.

Author Contributions: Conceptualization, K.J.; methodology, T.Z. and R.L.; validation, R.L. and S.C.; formal analysis, T.Z., K.J. and R.L.; investigation, T.Z. and S.C.; resources, K.J.; writing—original draft preparation, K.J. and T.Z.; writing—review and editing, K.J.; supervision, K.J.; project administration, K.J.; funding acquisition, K.J. All authors have read and agreed to the published version of the manuscript.

Funding: This research is supported by the Department of Science and Technology of Guangdong Province (Project No. 2021QN02X239), National Natural Science Foundation of China (No. 62002394) and the Shenzhen Science and Technology Program (Grant No. 202206193000001, 20220817175048002).

Data Availability Statement: No new data were created or analyzed in this study. Data sharing is not applicable to this article.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of this study, in the collection, analyses, or interpretation of data, in the writing of the manuscript, or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

SPD	strongly Pigou–Dalton
MNW	Maximum Nash Welfare
USW	utilitarian social welfare

Appendix A. All Criteria in Example 1 Are SPD

Claim A1. Minimizing $\text{EnvySum}(\mathbf{p})$ is equivalent to minimizing $\text{GiniIndex}(\mathbf{p})$.

Proof.

$$\begin{aligned} \sum_{h_i < h_j} (h_j - h_i) &= \sum_{i < j} (h_j^\uparrow - h_i^\uparrow) \\ &= \sum_j (h_j^\uparrow (j-1)) - \sum_i (h_i^\uparrow (n-i)) \\ &= \sum_i (h_i^\uparrow (i-1-n+i)) \\ &= 2 \sum_i (i \cdot h_i^\uparrow) - (1+n) \sum_i (h_i^\uparrow) \\ &= 2 \text{GiniIndex}(\mathbf{p}) - (1+n)m. \end{aligned}$$

□

Lemma A1. Consider two profiles $\mathbf{p} = (p_1, \dots, p_n)$ and $\mathbf{q} = (q_1, \dots, q_n)$, where $\mathbf{q}$ is more balanced than $\mathbf{p}$ (namely, there are $j, k \in [n]$, such that $p_j > q_j \geq q_k > p_k$ and $q_i = p_i$ for $i \in [n] \setminus \{j, k\}$).

1. $\text{NSW}^\top(\mathbf{q}) < \text{NSW}^\top(\mathbf{p})$.
2. $\text{Potential}_\Phi(\mathbf{q}) < \text{Potential}_\Phi(\mathbf{p})$, where $\Phi(x)$ is a **strictly** convex function of x . (Hence criteria $\sum_i \Phi(h_i)$ with strictly convex terms $\Phi(h_i)$, including $\text{Entropy}^\top(\mathbf{p})$, $\text{Congestion}(\mathbf{p})$, $\text{LexiMax}(\mathbf{p})$, and $\text{LexiMin}(\mathbf{p})$ are all SPD.)
3. $\text{EnvySum}(\mathbf{q}) < \text{EnvySum}(\mathbf{p})$.
(Hence $\text{GiniIndex}(\mathbf{q}) < \text{GiniIndex}(\mathbf{p})$ due to Claim A1.)

Therefore all criteria mentioned in Example 1 are SPD.

Proof. By definition, we assume $q_j = p_j - \Delta$ and $q_k = p_k + \Delta$, where $\Delta > 0$. Then we have $p_j - p_k \geq 2\Delta$.

1.

$$\begin{aligned} \text{NSW}^\top(\mathbf{q}) - \text{NSW}^\top(\mathbf{p}) &= -(q_j q_k - p_j p_k) \prod_i p_i \\ &= (p_j p_k - (p_j - \Delta)(p_k + \Delta)) \prod_i p_i \\ &= (p_j p_k - p_j p_k - \Delta p_j + \Delta p_k + \Delta^2) \prod_i p_i \\ &= (\Delta(\Delta - (p_j - p_k))) \prod_i p_i \\ &\leq (\Delta(\Delta - 2\Delta)) \prod_i p_i \\ &= -(\Delta^2) \prod_i p_i < 0. \end{aligned}$$

Thus we have $\text{NSW}^\top(\mathbf{q}) < \text{NSW}^\top(\mathbf{p})$.

2.

$$\begin{aligned} \text{Potential}_\Phi(\mathbf{q}) - \text{Potential}_\Phi(\mathbf{p}) &= (\Phi(q_k) - \Phi(p_k)) + (\Phi(q_j) - \Phi(p_j)) \\ &= (\Phi(p_k + \Delta) - \Phi(p_k)) - (\Phi(q_j + \Delta) - \Phi(q_j)). \end{aligned}$$

Since $\Phi(x)$ is a strictly convex function of x , and $q_j > p_k$, we have

$$(\Phi(p_k + \Delta) - \Phi(p_k)) < (\Phi(q_j + \Delta) - \Phi(q_j)).$$

Thus,

$$\text{Potential}_\Phi(\mathbf{q}) < \text{Potential}_\Phi(\mathbf{p}).$$

3. Let n_1, n_2, n_3, n_4 and n_5 be the number of agents i except j, k with $p_i \leq p_k$, $p_k < p_i \leq q_k$, $q_k < p_i < q_j$, $q_j \leq p_i < p_j$ and $p_i \geq p_j$ respectively.

Let $U(\mathbf{p}, i) = \sum_{h_i < h_j} (h_j - h_i)$ denote the unhappiness of agent i . Then we have

$$\text{EnvySum}(\mathbf{p}) = \sum_i U(\mathbf{p}, i),$$

$$\text{EnvySum}(\mathbf{q}) - \text{EnvySum}(\mathbf{p}) = \sum_i (U(\mathbf{q}, i) - U(\mathbf{p}, i)).$$

Observe the change from $\mathbf{p}$ to $\mathbf{q}$, we derive that:

- (i) For those agents i with $p_i \leq p_k$ or $p_i \geq p_j$, their $U(\mathbf{p}, i) = U(\mathbf{q}, i)$ s remain unchanged.
- (ii) For an agent i with $p_k < p_i \leq q_k$, its $U(\mathbf{p}, i)$ changes by

$$U(\mathbf{q}, i) - U(\mathbf{p}, i) = (q_k - p_i + q_j - p_i) - (p_j - p_i) = q_k - p_i - \Delta \leq 0.$$

- (iii) For those agents i with $q_k < p_i < q_j$, their total sum of unhappiness is changed by

$$\Delta_3 = \sum_{q_k < p_i < q_j} (U(\mathbf{q}, i) - U(\mathbf{p}, i)) = -n_3 \Delta.$$

- (iv) For an agent i with $q_j \leq p_i < p_j$, its unhappiness changes by

$$U(\mathbf{q}, i) - U(\mathbf{p}, i) = -(p_j - p_i) \leq 0.$$

- (v) For j , its unhappiness changes by

$$\Delta_j = U(\mathbf{q}, j) - U(\mathbf{p}, j) \leq (n_4 + n_5) \Delta,$$

because of the decrease from p_j to q_j .

- (vi) For k , its unhappiness changes by

$$\Delta_k = U(\mathbf{q}, k) - U(\mathbf{p}, k) \leq -(n_3 + n_4 + n_5) \Delta - 2\Delta.$$

The $-(n_3 + n_4 + n_5) \Delta$ part is due to the increase from p_k to q_k , and the remaining -2Δ part is due to the increase from p_k to q_k and the decrease from p_j to q_j .

Since case (i), (ii) and (iv) contribute non-positive terms in $\text{EnvySum}(\mathbf{q}) - \text{EnvySum}(\mathbf{p})$, we may only consider (iii), (v) and (vi) and have:

$$\begin{aligned} & \text{EnvySum}(\mathbf{q}) - \text{EnvySum}(\mathbf{p}) \\ &= \sum_i (U(\mathbf{q}, i) - U(\mathbf{p}, i)) \\ &\leq \Delta_3 + \Delta_j + \Delta_k \\ &\leq -n_3 \Delta + (n_4 + n_5) \Delta - (n_3 + n_4 + n_5) \Delta - 2\Delta \\ &= -n_3 \Delta - 2\Delta < 0. \end{aligned}$$

Thus we have $\text{EnvySum}(\mathbf{q}) - \text{EnvySum}(\mathbf{p}) < 0$. $\square$

Appendix B. Inconsistency for the Ternary or Item-Dependent Valuations

When the valuations of agents are not binary but ternary ($v_i(\{o_j\}) \in \{0, 1, x\}$) or item-dependent ($v_i(\{o_j\}) \in \{0, p_j\}$), the optimal allocations under different SPD criteria may differ. In other words, the consistency of optimums among all the SPD criteria for the case of binary valuations does not generalize to the case of ternary or item-dependent valuations. We show an example in the following.

Suppose there are four agents and 10 items. The matrix v below represents the valuations of agents: for agent i the value of item j is $v_{i,j}$. In this example, the valuations can be seen as ternary with $x = 3$ or item-dependent with $p_1 = \dots = p_4 = 3$ and $p_5 = \dots = p_{10} = 1$.

$$v = \begin{pmatrix} 3 & 0 & 0 & 3 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 3 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

In this example, all allocations can be divided into two classes depending on whether agent 1 or agent 4 gets item 4. It is not hard to verify:

One optimal allocation for minimizing LexiMin admits profile $(6, 4, 4, 4)$. The bundles allocated to each agent in one of the LexiMin optimal allocation are $\{o_1, o_4\}$, $\{o_2, o_5\}$, $\{o_3, o_6\}$ and $\{o_7, o_8, o_9, o_{10}\}$.

One optimal allocation for minimizing LexiMax admits profile $(3, 5, 5, 5)$. The bundles allocated to each agent in one of the LexiMax optimal allocation are $\{o_1\}$, $\{o_2, o_5, o_6\}$, $\{o_3, o_7, o_8\}$ and $\{o_4, o_9, o_{10}\}$.

Appendix C. Inconsistency for the Mixed Case

When some items are divisible, and others are indivisible, the optimal allocations under different SPD criteria may differ. In other words, the consistency of optimums among all the SPD criteria for the divisible and indivisible cases, respectively, does not generalize to the mixed case. We show an example in the following. The agents are assumed to have 0/1-add valuations.

Suppose there are four agents and six items. The first four items are indivisible, whereas items 5, 6 are divisible. The matrix v below represents the valuations of agents: for agent i the value of item j is $v_{i,j}$:

$$v = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 1 & 1 \end{pmatrix}.$$

We use matrix b to represent the allocation, where $b_{i,j}$ indicates the amount of item j allocated to agent i . Note that $b_{i,j} = 0$ if $v_{i,j} = 0$ and the j th column of matrix b contains only one if item j is indivisible. The sum of each column of the matrix equals one, and the sum of each row equals the number of items obtained by each agent.

In this example, all allocations can be divided into two classes depending on whether item 4 is assigned to agent 1 or agent 4. It is not hard to verify:

The optimal allocation for minimizing LexiMin is shown in b_{LexiMin} below, admitting profile $(2, \frac{4}{3}, \frac{4}{3}, \frac{4}{3})$.

The optimal allocation for minimizing LexiMax is shown in b_{LexiMax} below, admitting profile $(1, \frac{5}{3}, \frac{5}{3}, \frac{5}{3})$.

$$b_{\text{LexiMin}} = \begin{pmatrix} 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & \frac{1}{3} & 0 \\ 0 & 0 & 1 & 0 & 0 & \frac{1}{3} \\ 0 & 0 & 0 & 0 & \frac{2}{3} & \frac{2}{3} \end{pmatrix}, b_{\text{LexiMax}} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & 1 & 0 & \frac{1}{3} & \frac{1}{3} \\ 0 & 0 & 0 & 1 & \frac{1}{3} & \frac{1}{3} \end{pmatrix}.$$

References

1. Coulter, P.B. *Measuring Inequality: A Methodological Handbook*, 1st ed.; Routledge: London, UK, 1989. [\[CrossRef\]](#)
2. Darmann, A.; Schauer, J. Maximizing Nash product social welfare in allocating indivisible goods. *Eur. J. Oper. Res.* **2015**, *247*, 548–559. [\[CrossRef\]](#)
3. Atkinson, A.B.; Bourguignon, F. Introduction: Income Distribution Today. In *Handbook of Income Distribution*; Atkinson, A.B., Bourguignon, F., Eds.; Elsevier: Amsterdam, The Netherlands, 2015; Volume 2, pp. xvii–lxiv. [\[CrossRef\]](#)
4. Barman, S.; Krishnamurthy, S.K.; Vaish, R. Greedy Algorithms for Maximizing Nash Social Welfare. In Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, Stockholm, Sweden, 10–15 July 2018; pp. 7–13.
5. Kleinberg, J.; Rabani, Y.; Tardos, É. Fairness in routing and load balancing. In *Proceedings of the 40th Annual Symposium on Foundations of Computer Science (Cat. No. 99CB37039)*, New York, NY, USA, 17–19 October 1999; IEEE: Piscataway, NJ, USA, 1999; pp. 568–578.
6. Aziz, H.; Rey, S. Almost group envy-free allocation of indivisible goods and chores. *arXiv* **2021**, arXiv:1907.09279.
7. Benabbou, N.; Chakraborty, M.; Igarashi, A.; Zick, Y. Finding Fair and Efficient Allocations for Matroid Rank Valuations. *ACM Trans. Econ. Comput.* **2021**, *9*, 1–41. [\[CrossRef\]](#)
8. Moulin, H. *Axioms of Cooperative Decision Making*; Cambridge University Press: Cambridge, UK, 1991; Volume 15.
9. Halpern, D.; Procaccia, A.D.; Psomas, A.; Shah, N. Fair division with binary valuations: One rule to rule them all. In *Proceedings of the Web and Internet Economics: 16th International Conference, WINE 2020, Beijing, China, 7–11 December 2020*; Proceedings 16; Springer: Berlin/Heidelberg, Germany, 2020; pp. 370–383.
10. Babaioff, M.; Ezra, T.; Feige, U. Fair and truthful mechanisms for dichotomous valuations. In Proceedings of the AAAI Conference on Artificial Intelligence, Virtually, 19–21 May 2021; Volume 35, pp. 5119–5126.
11. Nace, D.; Doan, L.N. *A Polynomial Approach to the Fair Multi-Flow Problem*; University of Technology of Compiègne: Compiègne, France, 2002.
12. Lin, Y.; Li, W. Parallel machine scheduling of machine-dependent jobs with unit-length. *Eur. J. Oper. Res.* **2004**, *156*, 261–266. [\[CrossRef\]](#)
13. Ibaraki, T.; Katoh, N. *Resource Allocation Problems: Algorithmic Approaches*; MIT Press: Cambridge, MA, USA, 1988.
14. Michaeli, I.; Pollatschek, M.A. On Some Nonlinear Knapsack Problems. *Ann. Discret. Math.* **1977**, *1*, 403–414.
15. Golovin, D. *Max-Min Fair Allocation of Indivisible Goods*; School of Computer Science, Carnegie Mellon University: Pittsburgh, PA, USA, 2005.
16. Bezáková, I.; Dani, V. Allocating indivisible goods. *SIGecom Exch.* **2005**, *5*, 11–18. [\[CrossRef\]](#)
17. Bansal, N.; Sviridenko, M. The santa claus problem. In Proceedings of the Thirty-Eighth Annual ACM Symposium on Theory of Computing, Seattle, WA, USA, 21–23 May 2006; pp. 31–40.
18. Edmonds, J.; Karp, R.M. Theoretical Improvements in Algorithmic Efficiency for Network Flow Problems. *J. ACM* **1972**, *19*, 248–264. [\[CrossRef\]](#)
19. Oxley, J. *Matroid Theory*; Oxford University Press: Oxford, UK, 2011. [\[CrossRef\]](#)

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.